APACHE PYSPARK

Complete Reference Guide

Architecture · Transformations · Memory Management · Optimization · Streaming

venkataharishbandha

Connect on LinkedIn • linkedin.com/in/venkataharishbandha

Covers All Major Topics:

Why Spark | Architecture | RDDs | DAG | Lazy Evaluation | Joins | Catalyst Optimizer | Memory Management | OOM & Skewness | Caching | Partition Pruning | AQE | DPP | Deployment Modes | Structured Streaming | Optimization

1. Why Apache Spark?

Apache Spark was built to overcome the limitations of Hadoop MapReduce:

- **Speed** → Hadoop relied heavily on disk I/O, while Spark performs in-memory computation, making it up to 100x faster.
- **Flexibility** → Spark supports batch, streaming, SQL, machine learning, and graph processing in one ecosystem.
- **Ease of Use** → Provides APIs in Python, Scala, Java, and R with simple transformations/actions instead of verbose MapReduce code.
- **Fault Tolerance** → Spark uses lineage & DAG to recompute data if something fails, ensuring reliability.

What is Apache Spark?

Apache Spark is an open-source, distributed computing system designed for big data processing and analytics.

- It provides an interface for programming entire clusters with implicit data parallelism and fault tolerance.

Known for its speed, ease of use, and versatility, Spark can handle:

- Batch Processing
- Real-time Streaming
- Machine Learning
- Graph Computations
- Interactive SQL Queries

2. Hadoop (Monolithic) vs Spark (Distributed)

Visual Data Flow Comparison

Hadoop (Monolithic MapReduce):

> [Input Data] → [Map] → [Write to Disk] → [Shuffle/Sort] → [Write to Disk] → [Reduce] → [Output] Every stage writes to disk → slow & I/O heavy.

Spark (Distributed):

> [Input Data Partitioned] → [Executors process in Memory] → [RDDs/DataFrames] → [DAG Optimization] → [Final Result] Data stays in memory as much as possible → fast & efficient.

Aspect	Hadoop (MapReduce)	Apache Spark
Architecture	Monolithic batch system, tightly coupled with HDFS	Distributed, modular ecosystem (SQL, Streaming, MLlib, GraphX)
Processing	Batch only – processes data in chunks	Supports Batch + Real-time + Interactive queries + ML + Graph
Speed	Disk-based I/O after each Map/Reduce stage → slow	In-memory computation, up to 100x faster

Aspect	Hadoop (MapReduce)	Apache Spark
Ease of Use	Complex Java MapReduce programs	Simple APIs in Python, Scala, Java, R, SQL
Fault Tolerance	HDFS replication (storage-heavy)	RDD lineage + DAG recomputation (efficient)
Scalability	Horizontal scaling, but heavy disk overhead	Native partitioning, parallelism across executors
Streaming	Not supported natively (needs Storm/Flink)	Built-in Structured Streaming
Ecosystem	Hive, Pig, Oozie (loosely integrated)	Unified ecosystem: Spark SQL, MLlib, GraphX, Streaming

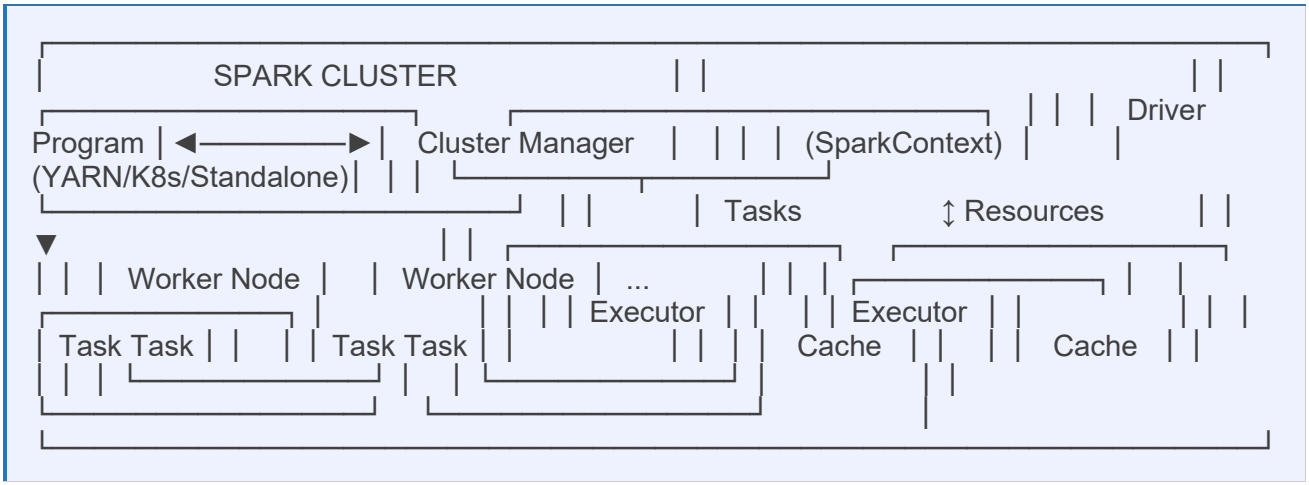
Summary
Hadoop = Monolithic, batch-only, disk-heavy. Spark = Distributed, multi-workload (batch + streaming + ML + SQL), memory-optimized.

3. Spark Architecture

Spark architecture is designed around a master-slave architecture, where a master (Resource Manager) orchestrates tasks and slaves (worker nodes/executors) perform the actual work.

Key components: Driver Program, SparkContext, Cluster Manager, Executors, Tasks, RDDs, DAG Scheduler, and Task Scheduler.

Architecture Diagram (Text Representation)



Component Breakdown

Driver Program

- **Role:** The main process that runs the main() function of your Spark application.

- Creates the SparkContext and coordinates task execution across the cluster.
- Defines datasets and operations (transformations & actions).
- **AMC (Application Master Container):** Created by Resource Manager on the Driver Node. Contains PySpark Main, JVM Main, and py4j.
- **py4j:** Bridge that converts PySpark code into JVM bytecode for execution.

SparkContext

- The entry point to Spark functionality, representing a connection to a Spark cluster.
- Used to create RDDs, accumulators, and broadcast variables.
- In modern Spark (2.0+), SparkSession has replaced and embedded SparkContext.

Cluster Manager

- **Allocates resources** (CPU, memory) across the cluster.
- **Supported managers:** Spark Standalone, Hadoop YARN (most popular), Mesos, Kubernetes.

Executors

- Worker node processes that run individual tasks in a Spark job concurrently.
- Perform tasks assigned by the driver and provide in-memory storage for RDDs.
- Each Worker Node has a JVM machine to execute tasks.
- If UDFs are written in Python, a Python interpreter/worker is also installed.

Tasks

- The smallest unit of work in Spark, representing transformations applied to partitions of data.
- Each task is executed in a separate thread within an executor.

DAG Scheduler

- Divides operator graphs into stages and sends tasks to the Task Scheduler.
- Translates data transformations into a physical execution plan.
- Optimizes the plan by rearranging and combining operations, grouping them into stages.

Task Scheduler

- Launches tasks via the cluster manager.
- Receives tasks from the DAG Scheduler and launches them on executor JVMs.
- Tasks for each stage launched in parallel — one per partition.

Spark Application Flow

1. Submit code to the Resource Manager.
2. The Resource Manager creates the Driver Program.
3. The Driver Program requests executors from the Cluster Manager.
4. The Cluster Manager creates the executors.
5. Executors connect to the Driver Program, receive tasks, execute them, and return results.

4. SparkContext vs SparkSession

SparkContext (Legacy)

- **Main entry point (legacy)** → Represents the connection between Driver and Cluster Manager.

- Responsibilities: coordinate & monitor task execution, request resources, create RDDs, Accumulators, Broadcast variables.
- Driver Program instantiates SparkContext when a Spark job starts.
- Original entry point in Spark (pre-2.0).

SparkSession (Modern — Recommended)

- **Unified entry point** → Introduced in Spark 2.0+, recommended for all new apps.
- Encapsulation: Combines SparkContext, SQLContext, HiveContext into one.
- APIs: Provides high-level APIs (DataFrame, SQL, Streaming, MLlib, GraphX).
- Automatic creation: In environments like Databricks, a SparkSession is auto-created.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName('MyApp') \
    .master('local[*]') \
    .getOrCreate()

sc = spark.sparkContext  # Access underlying SparkContext
```

Aspect	SparkContext	SparkSession
Usage	Legacy entry point	Modern, unified entry point
Scope	Low-level (RDDs, accumulators, broadcast vars)	High-level + Low-level (DataFrame, SQL, Streaming, MLlib, GraphX)
Encapsulation	Exists independently	Wraps SparkContext, HiveContext, SQLContext
Introduction	Pre-Spark 2.0	Spark 2.0+
Access	Created manually in Driver	Created via SparkSession.builder or auto-provided (Databricks)

In essence
SparkContext = Core engine connector (still exists under the hood). SparkSession = Developer-friendly, unified API (recommended).

Application Master Container (AMC) in Spark

Spark integrates with cluster managers like YARN using the Application Master Container (AMC), which hosts the Driver and orchestrates execution.

- **AMC:** Created by the Resource Manager on the designated Driver Node. Responsible for all driver program activities.
- **JVM Main** → Core driver for Scala/Java code.
- **PySpark Main** → Optional driver for Python code.
- **py4j** → Converts PySpark code into JVM bytecode for execution.

- **Driver Program (inside AMC):** Runs main(), initializes SparkSession, translates code into execution plans (DAG), requests Executors. If Driver fails, entire Spark job fails.

Deployment Modes

- **Client Mode** → Driver runs on the machine submitting the job; used for interactive/debugging.
- **Cluster Mode** → Driver runs inside AMC on the cluster; preferred for production — better fault tolerance and network efficiency.

5. Lazy Evaluation and Actions in Apache Spark

Spark operations are divided into Transformations and Actions, governed by the principle of lazy evaluation.

5.1 Lazy Evaluation

- **Transformations** create a new DataFrame (or RDD) from an existing one — e.g., filter(), select(), groupBy(), join().
- Calling a transformation does NOT trigger computation immediately.
- Instead, Spark builds a logical execution plan known as a Directed Acyclic Graph (DAG).
- This deferred execution allows Spark to optimize the entire plan by rearranging and merging transformations for better performance before any data processing begins.

5.2 Actions

- Actions trigger the execution of the logical execution plan and either return results to the driver or write data to storage.
- An action acts as a "switch" to start Spark's computation.

Common actions:

- `df.show()` — Display first few rows
- `df.count()` — Count rows
- `display(df)` — Databricks-specific display command (also an action)
- `df.collect()` — Collect all rows to driver (use with caution on large data)
- `df.write.format(...).save(...)` — Write data to external sink

5.3 Demonstration (PySpark)

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder.appName('LazyEvalDemo').getOrCreate()

data = [(1, 'Alice', 'New York'), (2, 'Bob', 'London'), (3, 'Charlie', 'New York')]

schema = StructType([
    StructField('ID', IntegerType(), True),
    StructField('Name', StringType(), True),
    StructField('City', StringType(), True)
])

df = spark.createDataFrame(data, schema)
```

```
# Transformations (no job triggered yet)
df_filtered = df.filter(col('City') == 'New York')
df_selected = df_filtered.select('ID', 'Name')

# Action triggers job execution
display(df_selected)
```

Execution for the entire plan (createDataFrame → filter → select → display) happens only when the action (display) runs.

5.4 Query Plans and DAGs

- Each action triggers a Spark Job.
- `df.explain()` shows the physical execution plan — read from bottom to top.
- Spark may combine multiple operations (e.g., filter + select) into a single optimized scan and projection.
- **DAG (Directed Acyclic Graph):** Directed (data flows forward), Acyclic (no loops), Graph (nodes = operations, edges = data flow). Each job has its unique DAG tracking data flow from source to result.

6. Partitions and Resilient Distributed Datasets (RDDs)

6.1 Partitions

- **Logical Division:** Partitions are the fundamental units of data division in Spark, splitting a DataFrame or RDD into independent chunks.
- **Distributed Processing:** Each partition is processed independently and in parallel by executors across the cluster.
- **Typical Size:** Default partition size aligns with HDFS block size (~128 MB). Optimal sizes range from 100 MB to 200 MB.
- **Importance:** Proper partitioning maximizes parallelism and avoids skew, balancing workload among executors.

6.2 Resilient Distributed Datasets (RDDs)

- **Core Abstraction:** RDDs are immutable, distributed collections of data split into logical partitions.
- **Fault Tolerance via Lineage:** Every RDD transformation creates a new RDD and maintains a lineage (DAG) graph. If data is lost, Spark can recompute partitions from earlier transformations.
- **Immutability:** RDDs cannot be altered after creation — transformations produce new RDDs.
- **Distributed Parallelism:** RDD partitions reside across multiple cluster nodes and execute in parallel.
- **Relation to DataFrames:** DataFrames and Datasets are higher-level abstractions that convert down to RDDs for execution.

RDD Partition Diagram

List Data RDD (Distributed across Executors) [Row 1] → [Row 1, Row 2] = Partition 1 (Executor A) [Row 2] [Row 3, Row 4] = Partition 2 (Executor B) [Row 3] [Row 5, Row 6] = Partition 3 (Executor C) [Row 4] [Row 7, Row 8] = Partition 4

(Executor D) [Row 5] [Row 6] Each partition processed in parallel → massive speedup [Row 7] [Row 8]

7. Types of Transformations: Narrow vs Wide

Transformations in Spark are classified based on whether they require shuffling (data exchange) between partitions.

7.1 Narrow Transformations

Characteristics:

- Each input partition contributes to at most one output partition.
- No data shuffling or network I/O required.
- Executed locally on data partitions within the same executor.
- More efficient due to lack of expensive data movement.

Common Examples:

- `df.filter()` — filtering rows
- `df.select()` — selecting columns
- `df.withColumn()` — adding or transforming columns (local row logic)
- `map()`, `flatMap()` (on RDDs)

Narrow Transformation (no shuffle): Partition 1 → Partition 1' (same executor) Partition 2 → Partition 2' (same executor) Partition 3 → Partition 3' (same executor) explain() output shows: Scan, Filter, Project — NO Exchange step

7.2 Wide Transformations

Characteristics:

- Each input partition can contribute to multiple output partitions.
- Requires shuffling data across the network to group or redistribute data by key.
- Leads to expensive network I/O and disk writes.
- Often causes new stages in the Spark execution plan.

Common Examples:

- `df.groupBy()` — aggregations by key
- `df.join()` — joining multiple DataFrames
- `df.orderBy() / df.sort()` — sorting
- `df.repartition()` — explicit shuffling
- `df.distinct()`

Wide Transformation (shuffle required): Partition 1 → many output partitions (data moves across network) Partition 2 → many output partitions Partition 3 → many output partitions explain() output shows: Exchange hashpartitioning → shuffle occurring

Type	Shuffle	Examples	Performance
Narrow	No shuffle	filter, select, map, withColumn	Efficient — local processing
Wide	Full shuffle	groupBy, join, orderBy, repartition, distinct	Costly — network I/O + disk writes

8. Repartition vs Coalesce in Apache Spark

Both are transformations used to change the number of partitions in a DataFrame or RDD.

8.1 Repartition

- **Purpose:** Increase or decrease the number of partitions.
- **Shuffling:** Always triggers a full shuffle of data across the network. It is a wide transformation.
- **Benefit:** Helps balance data evenly across executors and can increase parallelism.
- **Cost:** More expensive due to network I/O and data shuffle.

```
df_repartitioned = df.repartition(numPartitions)
```

8.2 Coalesce

- **Purpose:** Reduce the number of partitions.
- **Shuffling:** Attempts to minimize or avoid shuffle by merging partitions on the same executor node.
- **Benefit:** Efficient way to reduce partitions without full shuffle, reducing overhead and number of output files.
- **Cost:** Less expensive than repartition but can create uneven partition sizes.

```
df_coalesced = df.coalesce(numPartitions)
```

Demonstration

```
df_initial = spark.createDataFrame([(1, 'A'), (2, 'B'), (3, 'C')], ['id', 'val'])
print(f'Initial partitions: {df_initial.rdd.getNumPartitions()}')

# Repartition (full shuffle)
df_repart = df_initial.repartition(3)
print(f'Repartitioned: {df_repart.rdd.getNumPartitions()}')  # 3
df_repart.explain()  # Shows Exchange (shuffle) step

# Coalesce (minimizes shuffle)
df_coalesce = df_repart.coalesce(1)
print(f'Coalesced: {df_coalesce.rdd.getNumPartitions()}')  # 1
df_coalesce.explain()  # May not show Exchange
```

Key Takeaway

Use repartition() when you need to INCREASE partitions or evenly redistribute data (shuffle is acceptable). Use coalesce() when REDUCING partitions efficiently without a costly shuffle.

9. Jobs, Stages, and Tasks in Apache Spark

Hierarchy

- **Job:** Represents the execution triggered by a single Spark action (e.g., count(), collect(), save()). Each action creates one or more jobs.
- **Stage:** A subset of a job based on the presence of shuffle boundaries. Comprises a sequence of narrow transformations that can be executed together. A wide transformation creates a boundary between stages.
- **Task:** The smallest unit of work sent to Executors. Each Task operates on a single partition of data. Number of Tasks in a stage = number of partitions in that stage.

Hierarchy Formula

Action (1) → Job (1) → Stages (1 to N) → Tasks (1 to N per Stage)

Diagram

Spark Application ├── Job 1 (triggered by: count()) │ ├── Stage 1 (narrow transforms: filter, select) │ │ ├── Task 1 (Partition 1) │ │ ├── Task 2 (Partition 2) │ │ └── Task N (Partition N) │ └── Stage 2 (wide: groupBy — shuffle boundary) │ ├── Task 1 (Partition 1) │ └── Task N (Partition N, default ~200) └── Job 2 (triggered by: write()) └── Stage 3 ...

Demonstration

```
from pyspark.sql.functions import col

# Disable AQE for clear observations
spark.conf.set('spark.sql.adaptive.enabled', False)

df_csv = spark.read.format('csv') \
    .option('header', True) \
    .option('inferSchema', True) \
    .load('/FileStore/tables/mega.csv')

# Narrow transformations (no shuffle, grouped in one stage)
df_narrow = df_csv.filter(col('product_name') == 'Sneakers') \
                  .select('order_id', 'product_name')

# Wide transformation causes new stage (shuffle involved)
df_wide = df_narrow.groupBy('product_name').agg({'order_id': 'count'})

# Action triggers job execution
display(df_wide)
```

Spark UI Observations

- Multiple jobs visible for the actions (reading with inferSchema, and final action).
- **Stage 1:** Narrow transformations (filter, select) with fewer tasks (based on partitions).
- **Stage 2:** Wide transformation (groupBy) involving shuffle, usually ~200 partitions/tasks unless changed.
- Shuffle Read/Write operations visible in DAG visualizations for wide transformations.

10. Joins in Spark

Join Mechanics Overview

- Joining two DataFrames is a wide transformation, requiring data shuffling.
- The shuffle ensures that all rows with the same join key (e.g., ID) reside in the same partition/executor.
- Spark applies a hash function on join keys (e.g., ID % 200) to distribute data into default 200 partitions.
- After shuffling, data is in a shuffled state where each partition contains matching records for its keys.

1. Shuffle Sort Merge Join (Default Strategy)

Process:

6. Shuffle data from both DataFrames by join key.
7. Sort data within each partition by join key.
8. Merge sorted partitions to produce joined results.

Characteristics:

- **Default for large DataFrames.** Efficient, scalable, and spill tolerant.
- **DAG Indicators:** Shows Scan, Exchange (shuffle), Sort, and SortMergeJoin stages.

2. Shuffle Hash Join

Process:

9. Shuffle data by join key.
10. Build in-memory hash table for smaller DataFrame per partition.
11. Scan larger side to find matches using the hash table.

Characteristics:

- Efficient when the smaller side fits in executor memory per partition.
- Avoids sorting step.

3. Broadcast Join (Broadcast Hash Join)

- **Purpose:** Avoids shuffle of the large DataFrame by broadcasting the small DataFrame to all executors.

Process:

12. Load small DataFrame into Driver memory.
13. Broadcast this small DataFrame to all executors.
14. Join happens locally in each executor.

Characteristics:

- Extremely fast for small dimension tables joined with large fact tables.
- Small DataFrame size typically < 5 MB (configurable).
- **DAG Indicator:** BroadcastExchange and BroadcastHashJoin stages — no shuffle.

Explicit Broadcast Join Code

```
from pyspark.sql.functions import broadcast

df_small = spark.createDataFrame([(1, 'DeptA'), (2, 'DeptB')], ['id', 'dept_name'])
```

```
df_large = spark.createDataFrame([(1, 'Item1'), (1, 'Item2'), (2, 'Item3')], ['id',
'item'])

df_joined = df_large.join(broadcast(df_small), on='id')
display(df_joined)
```

Join Type	When to Use	Characteristics
Sort Merge Join	Both DataFrames large; broadcast not feasible	Requires shuffle and sort; scalable and robust
Shuffle Hash Join	One side per partition is small but not broadcast-eligible	Avoids sorting; hash table built per partition
Broadcast Join	One DataFrame very small and fits in executor memory	Avoids shuffle of large table; fastest for small side

11. Spark SQL Engine / Catalyst Optimizer

The Spark SQL Engine uses the Catalyst Optimizer to convert declarative DataFrame/SQL code into an efficient physical execution plan. This process powers Spark's performance optimization.

Flow of Query Optimization

SQL / DataFrame / Hive Query ↓ 1. Unresolved Logical Plan (raw AST — not yet validated) ↓ [Analysis using Catalog] 2. Resolved Logical Plan (all columns/tables validated) ↓ [Rule-based Optimizations] 3. Optimized Logical Plan (predicate pushdown, projection pruning, constant folding) ↓ [Physical Planning] 4. Physical Plans (multiple candidates: sort merge, broadcast, hash join) ↓ [Cost Model] 5. Best Physical Plan selected ↓ 6. Code Generation → Execution on Executors → RDDs

Optimization Steps in Detail

15. **Unresolved Logical Plan:** Initial raw abstract syntax tree (AST). 'Unresolved' because not yet checked against Spark's metadata/catalog.
16. **Analysis (Using Catalog):** Spark Catalog stores metadata about databases, tables, columns, and types. Analyzes for column/table existence. Throws AnalysisException if issues arise.
17. **Resolved Logical Plan:** All references validated and linked to actual data sources.
18. **Optimized Logical Plan — Rule-based optimizations:**
    - **Predicate pushdown** — filters pushed early
    - **Projection pruning** — dropping unused columns
    - Combining transformations
    - Constant folding (precomputing constants)
19. **Physical Plans Generation:** Generates multiple candidate physical execution plans (e.g., different join strategies).
20. **Cost Model & Best Plan Selection:** Estimates resource usage (CPU, memory, I/O, network) for each plan. Chooses the least costly plan.
21. **Execution:** The selected physical plan runs on executors to process the data.

12. Driver Memory Management

Driver's Role

- **The Driver is the control center** of a Spark application.
- It coordinates task execution, holds metadata and DAGs, and manages broadcast variables.

Driver Memory Components

1. JVM Heap Memory (On-Heap Memory)

- Main memory area for JVM tasks in the Driver process.
- **Stores:** DAGs, metadata, task scheduling info, and broadcast variables (e.g., small DataFrame in broadcast join).
- **Configuration:** spark.driver.memory (e.g., --driver-memory 10g).

2. Overhead Memory (Off-Heap Memory)

- Memory reserved outside JVM heap for JVM threads, native code, and libraries.
- Ensures stability of the Driver process.
- **Default:** 10% of JVM heap OR 384 MB minimum (whichever is higher).
- Added automatically to total Driver memory.

Example	Heap	Overhead	Total Driver Memory
Example 1	10 GB	1 GB (10%)	11 GB
Example 2	1 GB	384 MB (minimum)	1.384 GB

Driver OOM Errors

Common causes:

- **Large Broadcast Variables:** Broadcasts exceeding JVM heap cause OOM in Driver.
- **Excessive df.collect() Usage:** Collecting entire datasets to Driver JVM heap can overflow memory.

Mitigation:

- `df.collect()` sparingly and only on small datasets or aggregated/single-row results.
- For large data, use distributed writes or sampling instead of full collection.

13. Executor Memory Management in Apache Spark

13.1 Executor Memory Components

1. JVM Heap Memory (On-Heap)

- **Configuration:** spark.executor.memory (e.g., --executor-memory 10g).
- Like the driver, 10% of this memory (or 384 MB, whichever higher) allocated for Overhead.

2. Off-Heap Memory

- Memory outside JVM heap (default 0 bytes). Useful for native code/experimental caching.

3. Overhead Memory

- Supports JVM threads, shared libraries, and native OS processes. Typically 10% of spark.executor.memory or 384 MB.

4. PySpark Memory

- Memory dedicated to PySpark for managing Python objects. Typically 0 bytes unless explicitly set.

13.2 JVM Heap Memory Breakdown

```
JVM Heap = spark.executor.memory (e.g., 10 GB)
│
├── Reserved Memory          = 300 MB (fixed, internal Spark ops)
│
├── Spark Memory Pool        = 60% of (Heap - 300 MB) ≈ 5.82 GB
│   ├── Storage Memory       = 50% of Spark Pool ≈ 2.91 GB  (cache/persist)
│   └── Execution Memory     = 50% of Spark Pool ≈ 2.91 GB  (shuffles/joins/sorts)
│       ↕ Dynamic boundary — can borrow from each other
│
└── User Memory              = 40% of (Heap - 300 MB) ≈ 3.88 GB (UDFs, custom structs)
```

13.3 Unified Memory Management (Spark Memory Pool)

- **Storage Memory:** Used for caching (df.cache() or persist()). Default 50% of Spark Memory Pool. Configured by spark.memory.storageFraction.
- **Execution Memory:** Used for shuffle buffers, joins, aggregations, sorting, and intermediate data. Default 50% of Spark Memory Pool.

Dynamic Boundary Rules

- Execution memory can borrow free Storage memory space.
- Execution memory can evict least recently used (LRU) Storage blocks as needed.
- Storage memory can borrow free execution memory but CANNOT evict execution memory.
- If Storage memory is full, it evicts its own cached data.

Component	Size (10 GB Heap)	Purpose
Reserved Memory	300 MB	Internal Spark operations, task scheduling
Spark Memory Pool	~5.82 GB	Shared by Storage and Execution
- Storage Memory	~2.91 GB	cache() / persist() — cached DataFrames
- Execution Memory	~2.91 GB	Joins, shuffles, aggregations, sorts
User Memory	~3.88 GB	UDFs, custom data structures, broadcast vars
Overhead Memory	~1 GB	JVM threads, native OS processes

14. Executor OOM Errors and Data Skewness

14.1 Executor Out of Memory (OOM)

- **Cause:** Occurs when a single partition's data size exceeds the available Execution Memory during processing (especially in wide transformations like groupBy or join).
- **Data Spilling:** Spark can spill intermediate data to disk if Execution Memory is full. However, it must spill partitions as whole units — cannot split partitions into smaller chunks.
- **Problem:** If a partition is larger than the executor's available Execution Memory, it cannot be fully processed or spilled → Executor OOM.

14.2 Data Skewness

- **Definition:** Data skew happens when certain keys (e.g., product_category='Food') have disproportionately large data volumes compared to others.
- **Impact:** In wide transformations (groupBy, join), all data for a skewed key ends up in one partition, creating a heavy 'hot' partition that can cause Executor OOM.

14.3 Handling Skewness with Salting

Instead of increasing executor memory (temporary and inefficient), the recommended approach is data skew elimination via Salting.

Salting Steps:

22. Identify the skewed column (e.g., product_category).
23. Decide the number of 'mini partitions' to split the skewed partition into (salt size).
24. Add a new column (salt) with random values (e.g., 0 to salt_size-1).
25. Perform wide transformation (groupBy or join) on both the original skewed column AND the new salt column.

Effect: Distributes the skewed key's data into multiple smaller partitions (Food_0, Food_1...), each fitting into executor memory.

Aspect	Detail
Cause of Executor OOM	Single partition larger than Execution Memory
Skew Impact	Heavy 'hot' partitions overload executors
Spilling Challenge	Spark spills whole partitions — no splitting
Skew Handling	Use Salting to distribute skewed keys across partitions
Memory Scaling	Increasing memory alone is not scalable

15. Storage Levels (Cache and Persist)

Caching and persisting are crucial techniques to optimize performance by storing intermediate DataFrame/RDD results in memory or on disk for reuse.

15.1 Caching — cache()

- **Purpose:** Stores a DataFrame or RDD in executor's Storage Memory for faster access in subsequent operations.
- **Mechanism:** When cached, Spark executes the lineage once and stores partitions in memory. Later actions read directly from memory without recomputation.

- **Recommendation:** Cache only small-to-medium DataFrames reused multiple times. Caching very large DataFrames might cause spilling or OOM.

```
df_to_cache.cache()
# With cache(), df.explain() shows InMemoryTableScan
# Spark UI (Storage tab) shows cached DataFrames and their sizes
```

15.2 Persisting — persist()

- **Generalization:** persist() allows specifying custom storage levels beyond the default used by cache().
- `cache()` is shorthand for `persist(StorageLevel.MEMORY_AND_DISK)` (for DataFrames).

Storage Level	Description	Default for cache?
MEMORY_AND_DISK	RAM; spill to disk if insufficient; deserialized objects	Yes (DataFrames)
MEMORY_ONLY	RAM only; recompute partitions if memory insufficient	Yes (RDDs)
DISK_ONLY	Store only on disk	No
MEMORY_ONLY_2	Same as MEMORY_ONLY but replicate partitions twice	No
MEMORY_AND_DISK_2	Same as MEMORY_AND_DISK but replicate partitions twice	No
OFF_HEAP	Experimental off-heap storage, reduces GC	No

```
from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)

# Remove data from memory/disk:
df.unpersist()
```

Summary
Use cache() for simple, default memory-and-disk caching of DataFrames reused frequently. Use persist() when you need fine-grained control over storage level. Both cache and persist store data LAZILY — actual storage happens when an action triggers computation.

16. Edge Node and Deployment Modes

16.1 Edge Node

- **Purpose:** An Edge Node is a dedicated machine that acts as an intermediary between a developer and the Spark cluster.
- **Role:** Developers log into the edge node, submit their Spark applications from there, and the edge node communicates with the cluster manager.
- **Benefit:** Provides a controlled and secure point of access to the cluster.

16.2 Deployment Modes

Feature	Client Mode	Cluster Mode
Driver Location	Edge Node (client machine)	Worker Node within Spark cluster
Fault Tolerance	Low — if client fails, job fails	High — managed by cluster resource manager
Network Latency	High — driver communicates over network to executors	Low — driver and executors in same cluster network
Logging/Debugging	Easy — logs available locally on client machine	Complex — logs must be retrieved from cluster aggregation
Use Case	Development and testing, interactive debugging	Production environments — recommended

17. Partition Pruning (Static and Dynamic)

17.1 Static Partition Pruning

- **What:** Optimization where Spark reads only relevant partition folders from disk when a filter on the partition column is applied.
- Avoids unnecessary I/O and speeds up queries.

Partitions on Disk

When writing data, Spark can partition data by columns, creating hive-style directory structures:

```
output_data/department=IT/
output_data/department=HR/
output_data/department=Finance/
```

Directory-based partitioning enables Spark to skip folders irrelevant to a query filter.

```
# Write partitioned data
df.write.mode('overwrite') \
    .partitionBy('department') \
    .parquet('dbfs:/FileStore/output_data')

# Without filter: scans all partition folders
df_all = spark.read.format('parquet').load(base_path)

# With filter on partition column — PRUNING OCCURS
df_hr = spark.read.format('parquet').load(base_path) \
            .filter(col('department') == 'HR')
df_hr.explain()  # shows scan only of 'department=HR' files
```

Best Practices and Gotchas

- Use hive-style directory partitioning with partitionBy for automatic pruning support.
- Ensure consistent data types and column naming in writes and reads.
- **Avoid wrapping partition columns in functions** in filters — this disables pruning.
- Push partition column filters early in DataFrame or SQL queries for maximum pruning efficiency.

17.2 Dynamic Partition Pruning (DPP)

Dynamic Partition Pruning is an advanced optimization technique that extends static partition pruning. It dynamically applies filters on partitioned tables at runtime during join queries.

How DPP Works

- **Scenario:** DF_fact (large, partitioned) joined with DF_dim (smaller, filtered).
- Spark Catalyst Optimizer applies the filter on DF_dim first.
- Broadcasts the filtered keys to pruning logic on DF_fact.
- DF_fact partitions are dynamically pruned to read only matching partitions.

Conditions for DPP to Work

26. The filter column on the smaller table must match the partitioning column on the larger table.
27. The column involved should be part of the join condition.

```
# Assume df_fact is partitioned by 'department'
# Assume df_dim is not partitioned and filtered on 'HR'
df_joined = df_fact.join(
    df_dim.filter(col('department') == 'HR'),
    on='department',
    how='inner'
)
display(df_joined)
# In Spark UI: df_fact scan shows only HR department files
```

DPP Notes

Enabled by default in Spark 3.0+ (spark.sql.optimizer.dynamicPartitionPruning.enabled = true). Works transparently without code changes beyond writing efficient join and filter predicates. Especially effective in star schema workloads with large fact tables and small filtered dimension tables.

18. Adaptive Query Execution (AQE)

AQE, introduced in Apache Spark 3.0, is a runtime optimization framework that dynamically adjusts query execution plans based on real-time statistics collected during query execution.

18.1 Key Features of AQE

1. Dynamically Coalesces Shuffle Partitions

- **Problem:** Fixed shuffle partitions (default 200) often cause small or empty partitions, leading to overhead.
- **AQE Solution:** Merges small shuffle partitions dynamically at runtime to create optimally sized partitions.
- **Benefit:** Reduces number of tasks, lowers GC pressure, improves resource utilization.

2. Dynamically Optimizes Join Strategy

- **Problem:** Traditional join strategies depend on static, compile-time statistics which can be inaccurate.
- **AQE Solution:** Collects runtime statistics and switches join strategies dynamically (e.g., Sort Merge → Broadcast Join if small table detected).

3. Dynamically Handles Skewness

- **Problem:** Data skew creates large partitions causing Executor OOM or slow processing.

- **AQE Solution:** Automatically detects skewed partitions at runtime and splits them into smaller sub-partitions.
- **Benefit:** Automates skew handling — no more manual salting required.

18.2 Enabling AQE

```
spark.conf.set('spark.sql.adaptive.enabled', 'true')
# or in spark-defaults.conf:
# spark.sql.adaptive.enabled=true

# Since Spark 3.2, AQE is ENABLED BY DEFAULT
```

Overall Impact

AQE makes Spark much more adaptive and performant by automating optimizations that previously required deep hands-on tuning. It enables Spark to handle data size variability, unpredictable skews, and evolving data characteristics with minimal user intervention.

19. PySpark Optimization Techniques

1. Lazy Evaluation

- Let Spark build a complete execution plan before running — enables full plan optimization.
- Avoid forcing early actions; chain transformations and trigger one action at the end.

2. Scanning Optimization with Partitions

- **Logical Partitions:** Spark creates logical partitions of data to distribute computation.
- **Default Size:** ~128 MB, aligning with HDFS block sizes. Config: spark.sql.files.maxPartitionBytes.
- **Partition Identification:** Use spark_partition_id() to add a column indicating partition IDs.
- **Repartitioning:** df.repartition(n) to match partitions with available cores/executors.
- **Partition Pruning:** Write data partitioned by columns, then filter on those columns to avoid full scans.

3. Joins Optimization Using Broadcasting

- Default Join (SMJ): shuffle + sort + merge — expensive due to network I/O.
- **Broadcast Join:** When one DataFrame is < 5-10 MB, broadcast it to all executors; eliminates shuffle of large DataFrame.

```
from pyspark.sql.functions import broadcast
df_joined = df_large.join(broadcast(df_small), on='join_key')
```

4. SQL Hints for Join Optimization

SQL hints provide guidance to Spark's Catalyst Optimizer:

```
SELECT /*+ BROADCAST(c) */
    *
FROM fact_table f
JOIN countries c ON f.country_id = c.country_id

-- Multiple hints:
SELECT /*+ BROADCAST(t1), MERGE(t2) */
FROM t1 JOIN t2 ON t1.key = t2.key
```

Spark hint priority: BROADCAST > MERGE > SHUFFLE_HASH > SHUFFLE_REPLICATE_NL

5. Caching and Persistence

- **Use cache():** For DataFrames reused frequently in downstream tasks.
- **Use persist():** For fine-grained control (disk-only, off-heap, etc.).
- **Use unpersist():** To explicitly free storage memory when done.

6. Dynamic Resource Allocation

- DRA allows Spark to automatically adjust the number of executors during runtime.
- Scales up when workload is high, releases executors when idle.

```
spark.conf.set('spark.dynamicAllocation.enabled', 'true')
spark.conf.set('spark.dynamicAllocation.executorIdleTimeout', '60s')
spark.conf.set('spark.dynamicAllocation.minExecutors', '1')
spark.conf.set('spark.dynamicAllocation.maxExecutors', '20')
spark.conf.set('spark.dynamicAllocation.initialExecutors', '2')
```

Property	Description	Typical Value
spark.dynamicAllocation.enabled	Enable dynamic allocation	true
spark.dynamicAllocation.executorIdleTimeout	Idle executor removal timeout	60s
spark.dynamicAllocation.minExecutors	Minimum executors maintained	0 or 1
spark.dynamicAllocation.maxExecutors	Maximum allowed executors	50
spark.dynamicAllocation.initialExecutors	Executors at startup	2

7. Broadcast Variables

- **Definition:** Read-only shared variables distributed and cached on each worker node.
- Instead of sending the variable with every task, Spark sends it once to each executor.
- Common use cases: lookup tables, small dimension tables, configuration data.
- **Suitable size:** Usually under 10 MB to fit in executor memory.

```
# Create broadcast variable
lookup_dict = {'A': 1, 'B': 2, 'C': 3}
broadcast_var = sc.broadcast(lookup_dict)

# Access in tasks
value = broadcast_var.value['A']
```

8. Accumulators

- **Definition:** Variables that support only associative and commutative operations like addition.
- Used to implement counters and sums efficiently over distributed tasks.
- Spark provides native numeric accumulators and supports custom accumulator types.

20. PySpark Structured Streaming

20.1 Introduction to Streaming & Evolution

Importance of Streaming

- Streaming is critical in modern big data applications, providing real-time insights as data continuously arrives from IoT, clickstreams, and sensors.
- Apache Spark excels by supporting both streaming and batch, enabling unified analytics pipelines.

Streaming vs Batch Processing

- **Batch Processing:** Handles fixed, bounded datasets — data with a clear beginning and end (e.g., historical reports).
- **Streaming:** Works with infinite, unbounded datasets — data flowing continuously (e.g., live logs, sensor data). Processes data as soon as it arrives.

DStreams (Deprecated)

- Early Spark Streaming API built upon RDDs.
- Cons: Low-level RDD-based API, poor SQL/DataFrame integration, separate code for streaming and batch.

Spark Structured Streaming (Current)

- **Unified API:** Same code for batch and streaming.
- **Concept:** Treats a stream as an unbounded table; every new data item is a row added to this table.
- **Benefits:** Reuse familiar DataFrame/SQL operations, easy switching between batch/streaming, simplifies fault tolerance.
- **Micro-batch:** Groups incoming data into small time-based intervals (e.g., 1-3 seconds) processed as Spark jobs.

20.2 Transformations in Streaming

Stateless Transformations

- **Concept:** Operate only on the current micro-batch of data, with no need to store or refer to previous data.
- **Examples:** select, filter, where, cast, map.
- Each micro-batch is processed independently. No history or state is maintained after each batch.

```
df = spark.readStream.format('rate').option('rowsPerSecond', 5).load()

# Stateless: filter even values, add derived column
result_df = df.filter(col('value') % 2 == 0).withColumn('is_even', lit(True))

query = result_df.writeStream \
    .outputMode('append') \
    .format('console') \
    .start()
query.awaitTermination()
```

Stateful Transformations

- **Concept:** Require maintaining and updating state between micro-batches.
- **Examples:** Aggregations (groupBy, count, sum), windowed computations, deduplication, streaming join.
- Executors maintain cached memory (state), updating it with each new batch.

```
df = spark.readStream.format('rate').option('rowsPerSecond', 5).load()
```

```
# Count events in a 10-minute window
agg_df = df.groupBy(window(df.timestamp, '10 minutes')).count()

query = agg_df.writeStream \
    .outputMode('complete') \
    .format('console') \
    .start()
```

20.3 Output Modes

Output Mode	Requirement	Behavior	Use Case
Append	Stateless only	Only new rows from each micro-batch appended	Logging, raw ingest, bronze layer
Complete	Stateful aggs	All current result rows written at every trigger	Dashboards, full aggregation views
Update	Stateful aggs	Only rows that changed since last batch are output	Change tracking, event-driven

20.4 Checkpoint Directory (Backbone of Streaming)

The checkpoint directory is fundamental for reliable and fault-tolerant streaming operations.

Component	Purpose
_spark_metadata	Stores query metadata — query ID, start timestamp, Spark version. Enables safe resume.
sources	Tracks files/data already read from source. Ensures files not reprocessed (idempotency).
offsets	Assigns offset/index to data already read. Indicates where to start next micro-batch.
commits	Markers for successfully processed micro-batches. Ensures at-least-once semantics.

```
df.writeStream \
    .option('checkpointLocation', '/mnt/checkpoints/path') \
    .start()
```

Important
Never manually modify checkpoint files or folders — doing so risks corruption and query inconsistencies.

20.5 Triggers in Structured Streaming

Trigger Type	Syntax	Behavior
Default	None required	Each batch runs immediately after previous completes.

Trigger Type	Syntax	Behavior
Processing Time	.trigger(processingTime='10 seconds')	Batches at fixed intervals. Near real-time.
Once (Deprecated)	.trigger(once=True)	Runs one batch for all available data, then stops.
Available Now	.trigger(availableNow=True)	Processes all current data in multiple micro-batches, then stops. Preferred.
Continuous (Exp.)	.trigger(continuous='1 second')	Row-by-row processing. Only append mode.

20.6 Windowing Operations (Time-Based Aggregations)

Event Time vs Processing Time

- **Event Time:** Timestamp of when event originally occurred (source time). Recommended for temporal accuracy.
- **Processing Time:** Timestamp when Spark processes the event. Can be delayed by system latency.

Types of Windows

- **Tumbling Window:** Fixed-size, non-overlapping. Each event belongs to exactly one window.
- **Sliding Window:** Fixed-size windows that overlap, with a slide interval smaller than window size.
- **Session Window:** Dynamic, non-fixed windows defined by bursts of activity and gaps.

```
from pyspark.sql.functions import window, col, count

# Tumbling Window
df_windowed = df_stream.groupBy(
    window(col('event_date'), '10 minutes'),
    col('color')
).agg(count('color').alias('count'))

# Sliding Window (10 min window, 5 min slide)
df_sliding = df_stream.groupBy(
    window(col('event_date'), '10 minutes', '5 minutes'),
    col('color')
).agg(count('color').alias('count'))

# Session Window
from pyspark.sql.functions import session_window
df_session = df_stream.groupBy(
    session_window(col('event_date'), '10 minutes'),
    col('color')
).agg(count('color').alias('count'))
```

20.7 Watermarking for Late Arrival Data

Watermarking prevents memory exhaustion in long-running streaming queries by bounding how long state is kept.

- **Growing State Problem:** Stateful operations require Spark to keep historical state. If held forever, memory grows unbounded.

- **Mechanism:** Watermark = Max Event Time Seen - Lateness Threshold. State ending before watermark is dropped. Late data older than watermark is discarded.

```
# Apply watermark on event time column
df_with_watermark = df_stream.withWatermark('event_date', '60 minutes')

# Windowed aggregation with watermarking
df_windowed = df_with_watermark.groupBy(
    window(col('event_date'), '10 minutes'),
    col('color')
).agg(count('color').alias('count'))
```

Output Mode	Compatible with Watermarking?	Notes
Update	Yes (best suited)	Outputs only updated rows; old state pruned
Append	Yes	Works without aggregations
Complete	No	Must output entire table — cannot prune state

21. Handling Failures and Optimization Best Practices

21.1 Failure Handling

Driver Failure

- Driver is the single point of failure for the Spark app — its crash terminates the entire application.
- **Solution:** Cluster managers like Kubernetes support driver supervision and restarts (spark.driver.supervise).

Executor Failure

- If an executor fails, running tasks are lost but Spark reschedules those tasks elsewhere.
- The app continues unless repeated task failures surpass the default retry limit (4).
- Prevent executor OOM by tuning resources and replicating data across nodes.

21.2 OOM Scenarios and Solutions

Driver OOM

- **Cause:** Excessive collect() pulling too much data. Use take(n) or limit data collected.
- **Cause:** Large broadcast variables exhausting driver memory. Avoid broadcasting very large datasets.
- **Fix:** Configure spark.driver.memory appropriately.

Executor OOM

- **Cause:** Large per-task output, wide transformations (groupByKey, big shuffles).
- **Fix:** Use reduceByKey/aggregateByKey over groupByKey. Unpersist unused DataFrames. Tune spark.memory.storageFraction.

21.3 Code and Resource Optimizations

Code-Level

- Prefer DataFrames/Datasets for optimized execution and memory savings.
- Use broadcast joins to reduce shuffle.
- Minimize shuffles by preferring reduceByKey over groupByKey.
- Avoid large collects.
- Manage partitions: coalesce small partitions, repartition too few partitions.
- Cache judiciously.

Resource-Level

- **spark.driver.memory** and **spark.executor.memory** — set according to node capacity.
- **spark.memory.fraction** (default 0.6) and **spark.memory.storageFraction** (default 0.5) — tune as needed.
- **spark.default.parallelism** and **spark.sql.shuffle.partitions** — control parallelism.
- Enable dynamic resource allocation (see Section 19.6).
- Use columnar formats (Parquet, Avro) for efficient analytics.

21.4 Resource Allocation Case Studies

Case 1: 6 Nodes, 16 Cores, 64 GB RAM each

```
Cores per Executor: 5 (optimal)
Executors per Node: (16 - 1 for OS) / 5 = 3
Total Executors: 6 nodes × 3 = 18 - 1 for YARN AM = 17
Memory per Executor: (64 GB - 1 GB OS) / 3 = ~21 GB
  Overhead = max(384 MB, 0.07 × 21 GB) = 1.47 GB
  Net Memory = 21 - 1.47 = ~19 GB

Final Config: 17 Executors | 5 Cores/Executor | 19 GB Memory/Executor
```

Case 2: 6 Nodes, 32 Cores, 64 GB RAM each

```
Cores per Executor: 5
Executors per Node: 32 / 5 ≈ 6
Total Executors: 6 × 6 = 36 - 1 for YARN AM = 35
Memory per Executor: (64 GB - 1 GB OS) / 6 ≈ 10 GB
  Overhead = 0.07 × 10 GB ≈ 0.7 GB (~1 GB)
  Net Memory = 10 - 1 = 9 GB

Final Config: 35 Executors | 5 Cores/Executor | 9 GB Memory/Executor
```

21.5 Best Practices for Spark Application Design

28. **Understand Data and Workload:** Know data size, skewness, and workload type.
29. **Design Logic:** Favor transformations, minimize shuffles, broadcast small datasets, use appropriate partitioning.
30. **Resource Configuration:** Set memory and cores carefully, enable dynamic allocation, manage parallelism.
31. **Infrastructure:** Use fast storage and networks, increase cores/nodes, optimize for data locality.
32. **Continuous Monitoring:** Use Spark UI for tracking resource utilization and bottlenecks; iterate tuning based on metrics.

22. Delta Lake Optimization

22.1 OPTIMIZE Command

- **Problem:** Frequent batch or streaming ingestion can produce many small files in Delta Lake tables. Reading many small files is inefficient due to high metadata overhead and increased I/O.
- **Solution:** The OPTIMIZE command compacts small files into fewer, larger files (~1 GB), reducing file count and improving query performance.
- **How it works:** Uses Delta Lake's transaction log to replace old small files with new compacted larger files.

```
-- SQL
OPTIMIZE table_name
OPTIMIZE table_name WHERE date >= '2022-11-18'

-- Python
from delta.tables import DeltaTable
deltaTable = DeltaTable.forName(spark, 'table_name')
deltaTable.optimize().executeCompaction()
```

22.2 ZORDER BY Command

- **Purpose:** Works with OPTIMIZE to improve query speed by colocating related data in the same files.
- **How:** Applies Z-order sorting based on specified columns. Uses column statistics (min/max) from Delta transaction log for data skipping.
- **Benefit:** Queries filtering on Z-ordered columns only scan files with overlapping filter value ranges, dramatically reducing I/O.

```
OPTIMIZE table_name ZORDER BY (customer_id)

-- Relationship: OPTIMIZE compacts file sizes,
--               ZORDER arranges data within those files
```

23. Important Points & Advanced Topics

23.1 Read & Write Operations — Transformations or Actions?

Read Operations

- **Lazily evaluated:** If schema is explicitly provided, Spark delays execution until an action is called.
- **Eagerly evaluated:** If inferSchema is disabled → triggers a job to get column names (1 job). If inferSchema=True → two jobs: one for header, one to scan records.

Write Operations

- Always considered actions. Trigger actual computation and write results to external storage.

23.2 How Spark Handles Data Larger Than Cluster Memory

- **Data Partitioning:** Splits large datasets into partitions (~128 MB each). Each partition processed independently.
- **Loading:** Spark loads partitions sequentially, processing as many as fit in memory. Enables datasets larger than total memory.
- **Spill to Disk:** When memory insufficient during shuffles, Spark automatically spills data partitions to disk.

- **Sequential Processing:** Never needs to load entire dataset simultaneously — processes in small units.

23.3 Processing 1TB of Data — Example

> Data: 1 TB = 1,048,576 MB HDFS Block Size: 128 MB Total Blocks/Partitions: 1,048,576 / 128 = 8,192 Cluster: 20 Executors × 5 Cores = 100 parallel tasks Data per Batch: 100 tasks × 128 MB = 12,500 MB = 12.5 GB RAM per Executor per Batch: 12.5 GB / 20 = 0.625 GB Total Batches to process 1 TB: 1,024 GB / 12.5 GB ≈ 82 batches

23.4 Memory Full Due to Stateful Aggregation — Strategies

- **Scale Horizontally:** Add more worker nodes to increase total cluster memory.
- **State Expiration:** Use timeouts or state eviction policies with mapGroupsWithState to remove old keys.
- **Built-in Aggregations:** Employ built-in aggregations (count, sum, avg) or approximate algorithms (HyperLogLog, t-digest).
- **State Store Compression:** Enable compression: spark.sql.streaming.stateStore.compression.codec.
- **Checkpointing:** Saves state periodically for recovery.

23.5 File Source in Spark Structured Streaming

- Monitors specified directory for new files; processes files atomically added after streaming starts.
- Does NOT process files existing before streaming or files modified in-place.
- Schema inference at runtime: spark.sql.streaming.schemaInference.

Quick Reference — Key Configurations

Configuration	Default	Purpose
spark.driver.memory	1g	Driver JVM heap size
spark.executor.memory	1g	Executor JVM heap size
spark.memory.fraction	0.6	Fraction of heap for Spark Memory Pool
spark.memory.storageFraction	0.5	Fraction of Spark Pool for Storage
spark.sql.shuffle.partitions	200	Default partitions for wide transforms
spark.sql.files.maxPartitionBytes	128 MB	Max partition size when reading files
spark.sql.adaptive.enabled	true	Enable AQE (Spark 3.2+)

Configuration	Default	Purpose
spark.dynamicAllocation.enabled	false	Enable dynamic executor allocation
spark.sql.autoBroadcastJoinThreshold	10 MB	Max size for auto broadcast join
spark.sql.optimizer.dynamicPartitionPruning.enabled	true	Enable DPP
spark.driver.supervise	false	Auto-restart driver on failure